

Secondo Progetto

Studio delle performance di una CNN binaria

Tommasi Francesco

2026-06-13

Contents

1	Consegna	1
1.1	Resnet18	2
1.2	MobileNet_v2	2
2	Studio delle performance	2
3	Pesi messi a 0	3
3.1	Resnet18	4
3.2	MobileNet_V2	5
4	Pesi di default	6
4.1	Resnet18	6
4.2	MobileNet_V2	8
5	Robustezza	9
5.1	Resnet18	9
5.2	Mobilenet_v2	9
6	Conclusione	11

1 Consegna

Creazione di una rete neurale di tipo CNN (convoluzionale) che classifichi e quindi distingua immagini di cane e gatto. L'obiettivo è quindi studiare le performance della CNN in particolare l'accuratezza della rete e la loss function in base al modello utilizzato e ai parametri associati. Sono stati utilizzati Resnet18 e ModelNetv2.

1.1 Resnet18

Rete neurale basata su residual blocks. Ogni blocco applica normali convoluzioni standard (che guardano contemporaneamente colore e spazio dei pixel) e utilizza scorciatoie che prendono l'input iniziale e sommarlo direttamente all'output del blocco. Questo evita la dispersione di informazioni all'aumentare della profondità della rete. Ha circa 12 milioni di parametri, una dimensione del file .pth di 45 Mb e esegue 1.8 miliardi di operazioni di calcolo, è quindi una rete molto precisa ma anche molto pesante.

1.2 MobileNet_v2

E' una CNN basata su Inverted Residuals + Linear Bottlenecks. MobileNetV2 invece di fare un calcolo unico e pesante, divide la convoluzione in due passaggi: prima analizza lo spazio pixel per pixel (canale per canale, separatamente), e poi unisce i canali con una convoluzione 1×1 riducendo i calcoli e Inverted Residual Block che Al contrario di ResNet espande temporaneamente il numero di canali al centro del blocco per catturare più dettagli, e poi li restringe alla fine. Qui, tra le parti strette vengono messe le scorciatoie riducendo drasticamente la RAM usata. La rete contiene 17 Bottleneck Blocks (cioè layer) e ognuno contiene 3 sottolayer per espandere il canale, elaborare lo spazio e stringere i canali. 3.5 Milioni di parametri, 14 Mb di spazio occupato e 300 milioni di operazioni di calcolo, molto più leggera ed efficiente di resnet.

2 Studio delle performance

La rete studiata è allenata su un dataset di 22500 immagini con 2 modelli già preaddestrati. (<https://www.kaggle.com/datasets/bhaviikjadara/dog-and-cat-classification-dataset>). Il preaddestramento dei modelli consiste nel fatto che queste due reti hanno già imparato le caratteristiche e i pesi da associare ad ognuna per distinguere le immagini. Il dataset su cui sono state allenate è di circa 14 milioni di foto.

Entrambi le reti sono state create per lavorare sulle immagini, utilizzano filtri 3D (altezza larghezza pixel e colore) che scorrono lungo l'immagine e in base ai pattern registrati associano ad ogni pixel un numero, creando centinaia di feature maps (nel nostro caso 512) corrispondenti ad ogni caratteristica importante per la classificazione. Infine la testa della rete guarda il vettore delle 512 caratteristiche e fa un calcolo pesato associando un peso ai due output e prendendo la decisione binaria finale. Di seguito sono illustrati i modelli e i risultati associati ad ogni modello.

Le grandezze dei parametri sono :

- 32 per il batch poichè ho utilizzato una scheda video nvidia T4 in ambiente google colab e 32 è il compromesso ottimale, satura bene i core della GPU rendendo l'addestramento veloce.
- learning rate è a 0.001 perchè troppo alto rischiamo che la mia CNN faccia passi da gigante mancando i punti minimi invece troppo piccolo rischiamo di dover fare mettere centinaia di epoche per avere una accuratezza alta impiegando molto tempo, con soltanto 5 epoche si fermerebbe ad una accuracy molto bassa (50-60%). E' importante sottolineare che questo passo è associato all'ottimizzatore Adam che adatta il passo dinamicamente in base all'andamento della loss function quindi il passo stesso non è sempre fisso ma è dinamico.
- 5 epoche perchè mettendo il learning rate a quel valore mi bastano solamente 5 epoche rendendo l'addestramento molto veloce (circa 10 min per entrambe su una GPU nvidia T4 oppure circa 3h su una CPU del mio portatile che ha 10 core fisici e 16 logici)

Nel mio studio ho analizzato le performance delle reti al variare di tre situazioni:

- La prima con i pesi del modello messi a 0 o meglio randomizzati

- La seconda con i pesi di default cioè quelli già imparati dalla rete quando è stata creata
- La terza situazione misura le performance della rete su un test eseguito su immagini rese più sporche, diminuendo la luminosità e ruotando le immagini.

3 Pesi messi a 0

Si svuotano i pesi di default di ogni modello mettendo *weights = None* quando definisco il modello (commentati in foto) :

```
def get_model(model_type="resnet18", num_classes=2):
    """
    Crea e modifica un modello pre-addestrato per la classificazione binaria.
    """
    if model_type == "resnet18":
        # Carichiamo ResNet18
        model = models.resnet18(weights='IMAGENET1K_V1')
        #model = models.resnet18(weights=None)
        num_fts = model.fc.in_features
        # Sostituiamo lo strato con uno nuovo che ha solo 2 output (Gatto/Cane)
        model.fc = nn.Linear(num_fts, num_classes)
        print("Modello ResNet18 configurato.")

    elif model_type == "mobilenet_v2":
        #model = models.mobilenet_v2(weights=None)
        model = models.mobilenet_v2(weights='DEFAULT')
        num_fts = model.classifier[1].in_features

        # Sostituiamo lo strato lineare finale
        model.classifier[1] = nn.Linear(num_fts, num_classes)
        print("Modello MobileNetV2 configurato.")
```

Figure 1: pesi = none

Facendo questo si cancella tutta la memoria dei pesi e si inizializza tutti i milioni di pesi con numeri casuali. Le prime feature maps generate saranno quindi senza senso perchè sono puro rumore casuale. Inoltre per questo tipo di pesi associato ad un learning rate basso peggiorerebbe ancora di più l'accuratezza e per risolvere il problema cioè avere un'accuratezza alta si dovrebbe aumentare di molto il numero di epoche oppure alzare il passo.

3.1 Resnet18

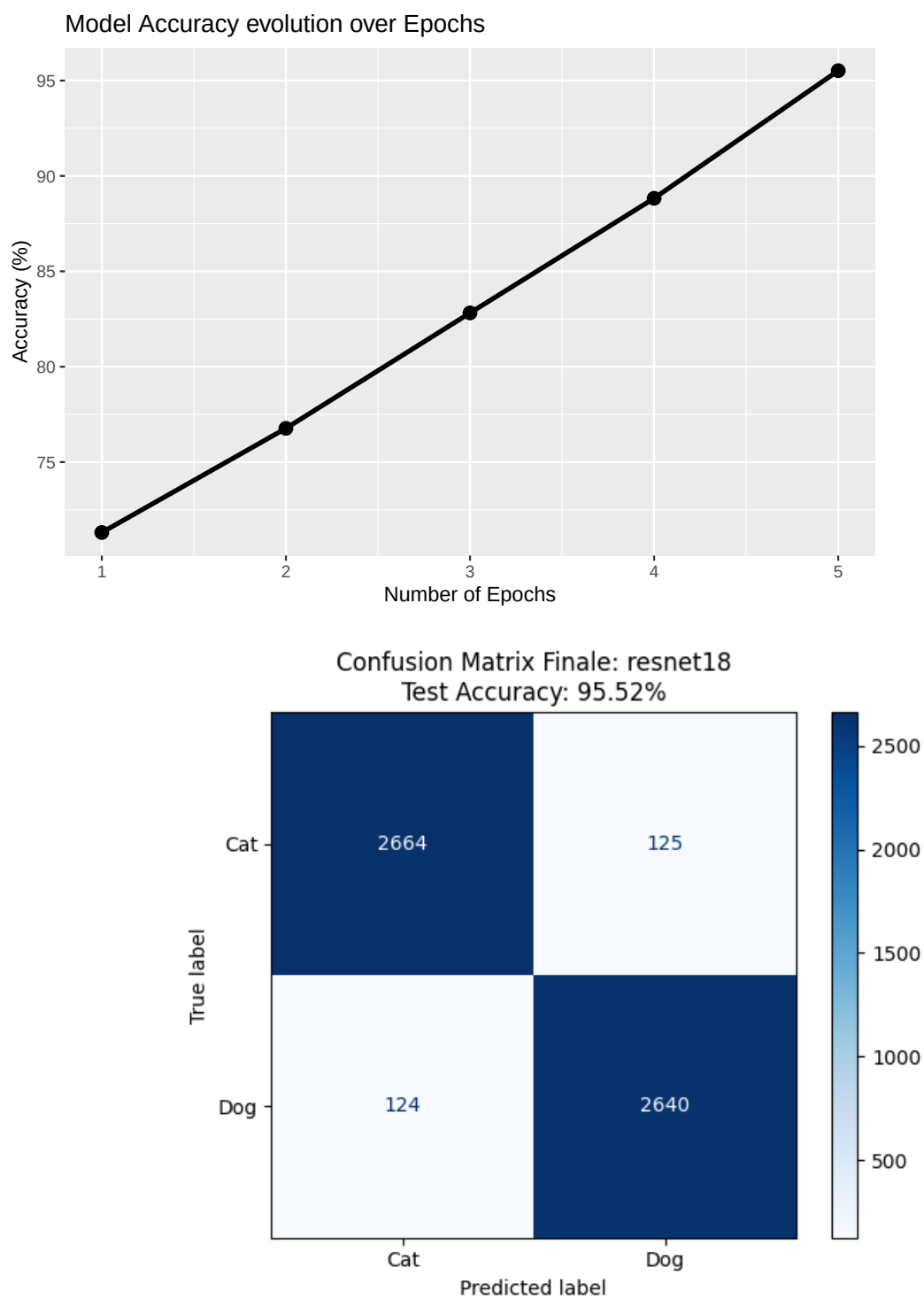
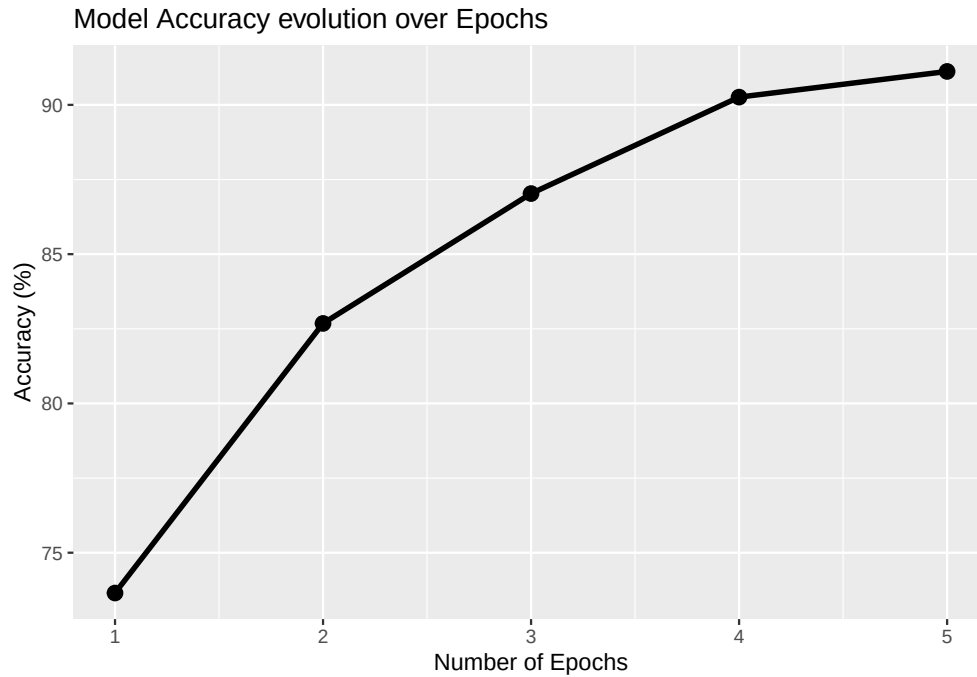
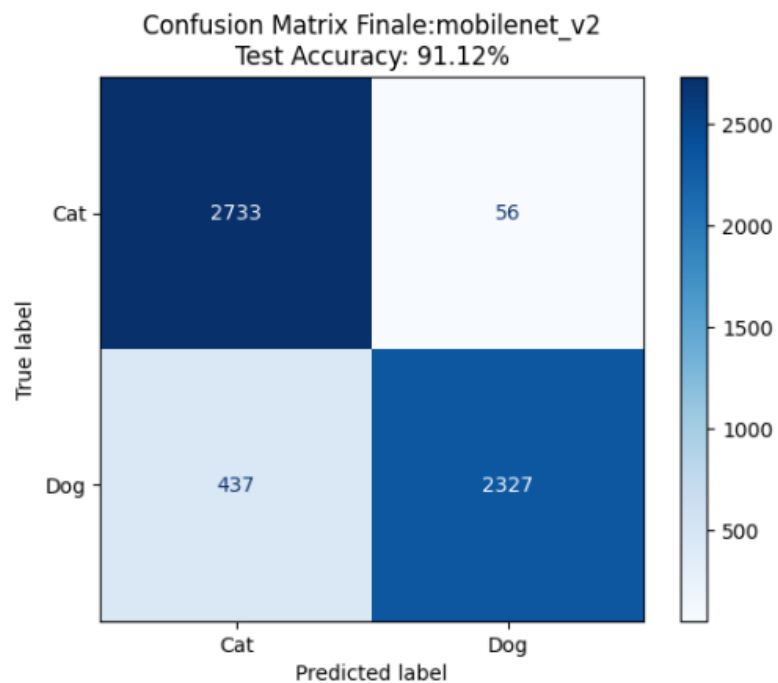


Figure 2: Resnet response

3.2 MobileNet_V2



il test associato:

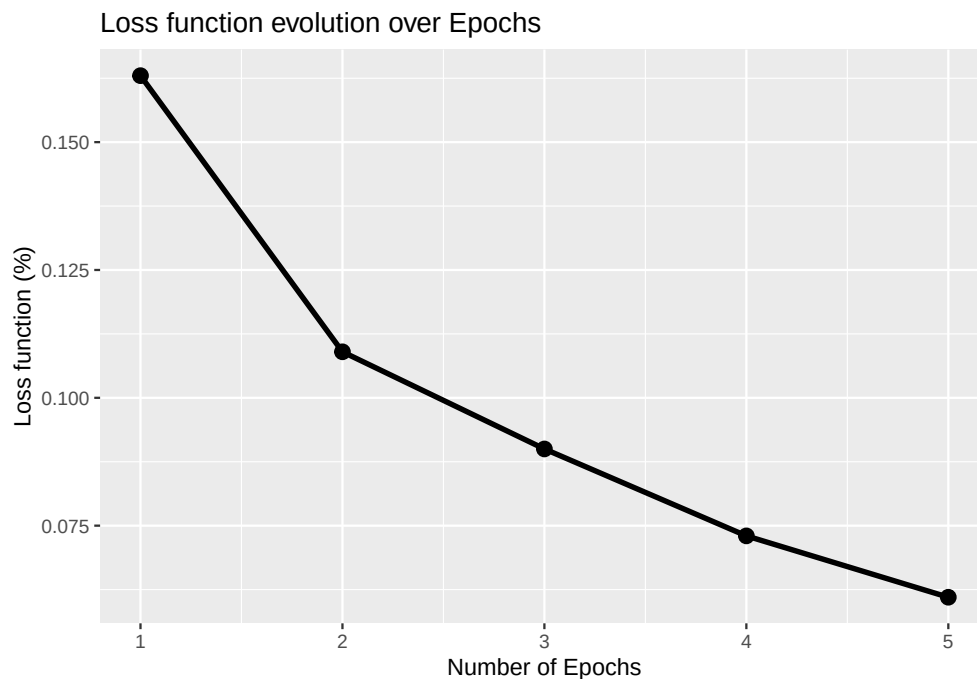
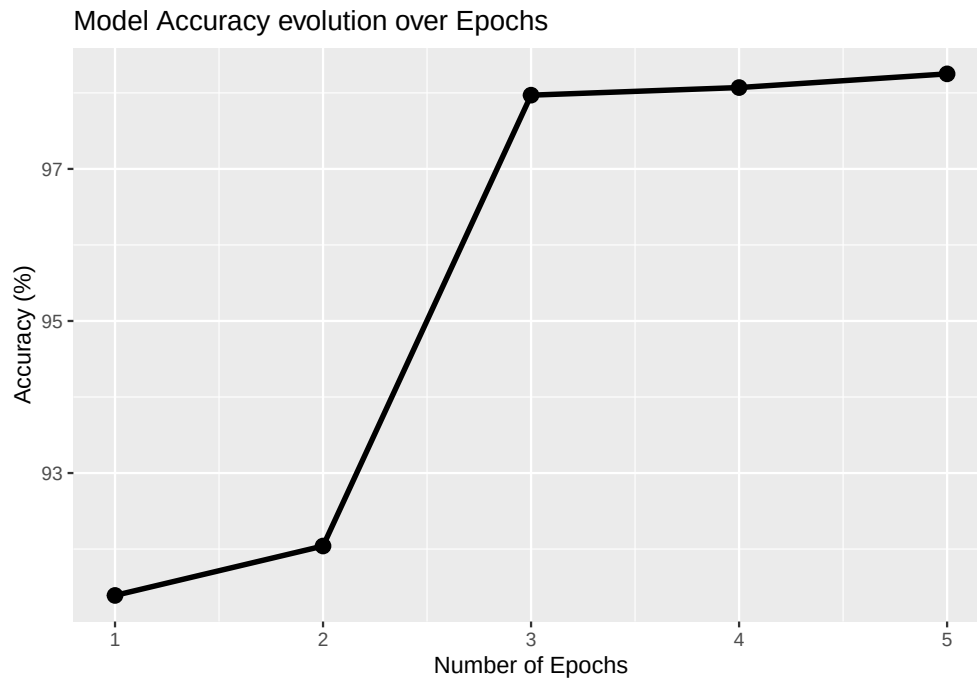


Come vediamo nei grafici l'accuratezza di entrambe i modelli parte da valori molto bassi, circa 70% perchè la prima epoca è quasi totalmente casuale la decisione binaria finale per poi crescere con le epoche, grazie all'allenamento.

4 Pesì di default

Nel codice, anche facendo una sola epoca o usando un learning rate minuscolo, la rete parte già da un'accuratezza altissima (98-99%). Questo perchè utilizzando i pesi già propri della rete l'addestramento serve solo a fare un piccolissimo “aggiustamento” (Fine-Tuning) per spiegare alla rete come associare quelle forme che già conosce ai due concetti di “Cane” e “Gatto”.

4.1 Resnet18



Loss che tocca lo 0.06

a fine training molto buona

Questa è la matrice di confusione associata al test di resnet18 . I risultati sono molto buoni con 98.33% di accuratezza per un dataset diverso da quello usato nel train.

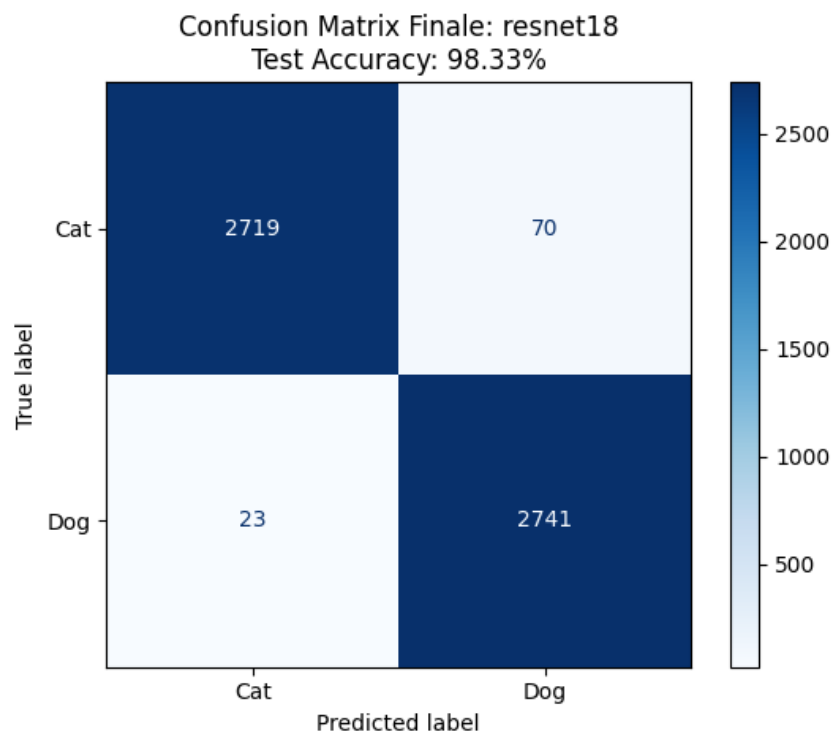
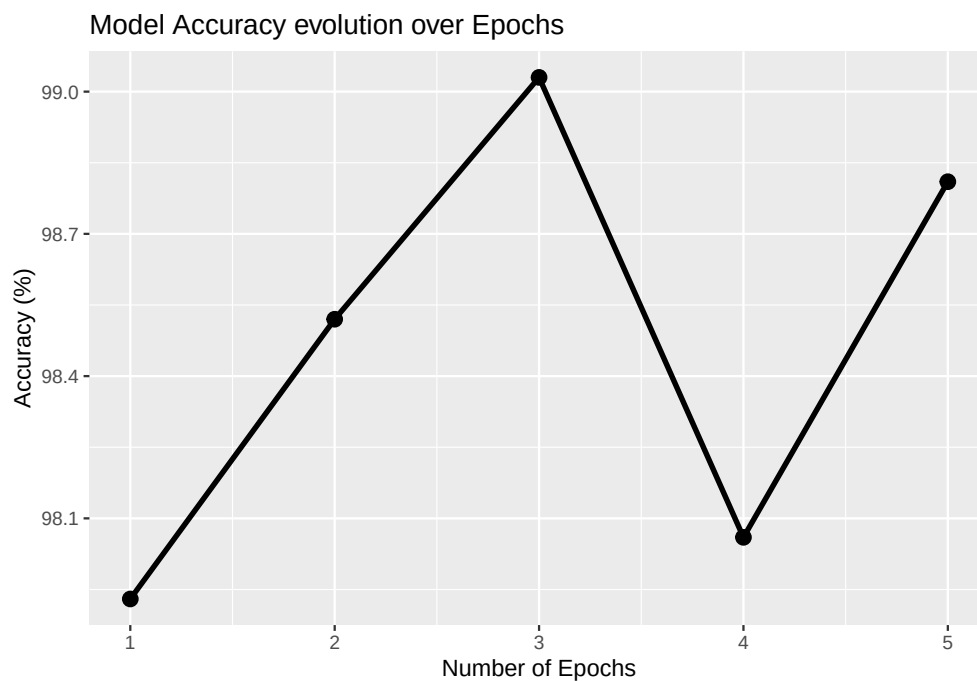
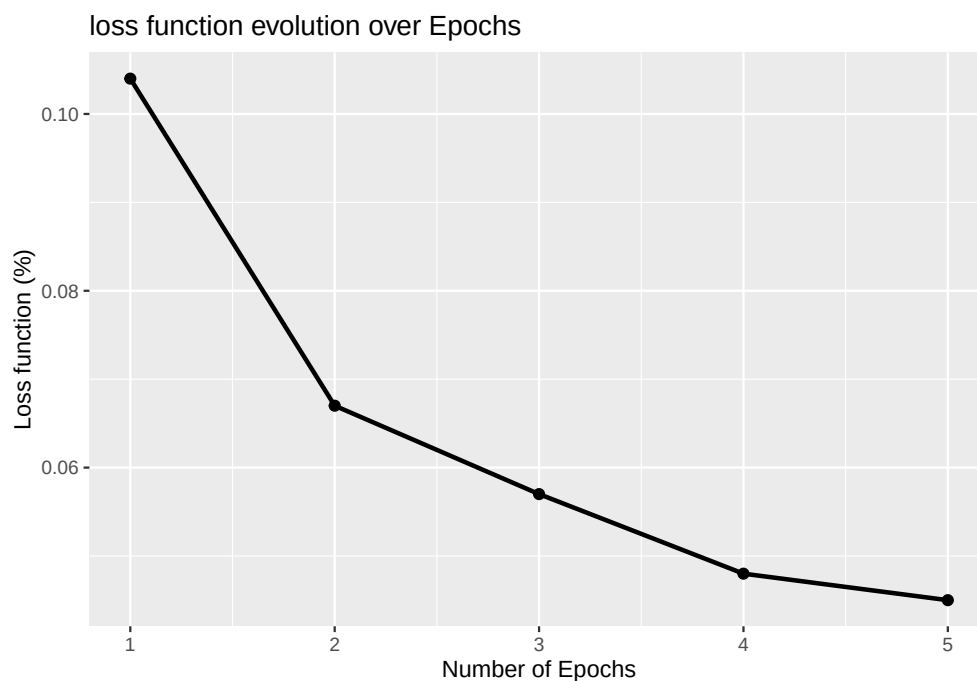


Figure 3: Resnet response

4.2 MobileNet_V2



l'accuratezza del modello ha un andamento che può sembrare strano in quanto cresce fino alla terza epoca per poi scendere leggermente (passa da 99.03 a 98.06 %). Questo potrebbe essere dovuto ad un leggero overfitting della rete e si potrebbe diminuire il passo (learning rate a 0.0001) per farla rallentare e fare in modo che si stabilizzi sui minimi tuttavia ha comunque una percentuale di accuratezza estremamente alta quindi decido di lasciarla così.



La loss function invece ha un andamento corretto cioè cala con l'aumentare delle epoche toccando il valore finale di 0.045.

Questa è la matrice di confusione associata al test di mobilenet_v2 .

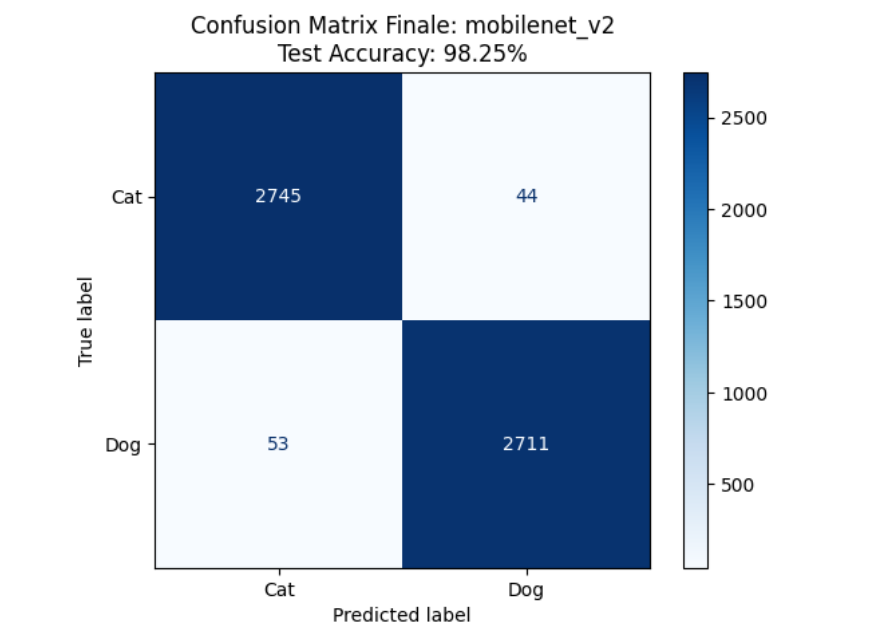


Figure 4: Mobilenet response

5 Robustezza

Come ultimo test ho analizzato la robustezza delle due reti testandole su un test_dataset in cui ruoto le foto di 45 gradi e diminuisco la luminosità in modo molto dinamico e direttamente sulla ram senza bisogno di dover creare un dataset a parte e caricarlo sul drive. La creazione del dataset dinamico distorto e più scuro è stata fatta con il blocco di codice che segue, creando solamente un altro blocco per il test e lasciandolo lavorare sui file .pth preesistenti con i pesi di default.

```
dirty_test_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomRotation(degrees=45),      # Ruota l'immagine in RAM
    transforms.ColorJitter(brightness=0.4, contrast=0.4), # Cambia la luce in RAM
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

Figure 5: This is my caption

I risultati del test con le immagini sporcate sono i seguenti

5.1 Resnet18

Risposta di resnet

5.2 Mobilenet_v2

Risposta di mobilenet praticamente identica

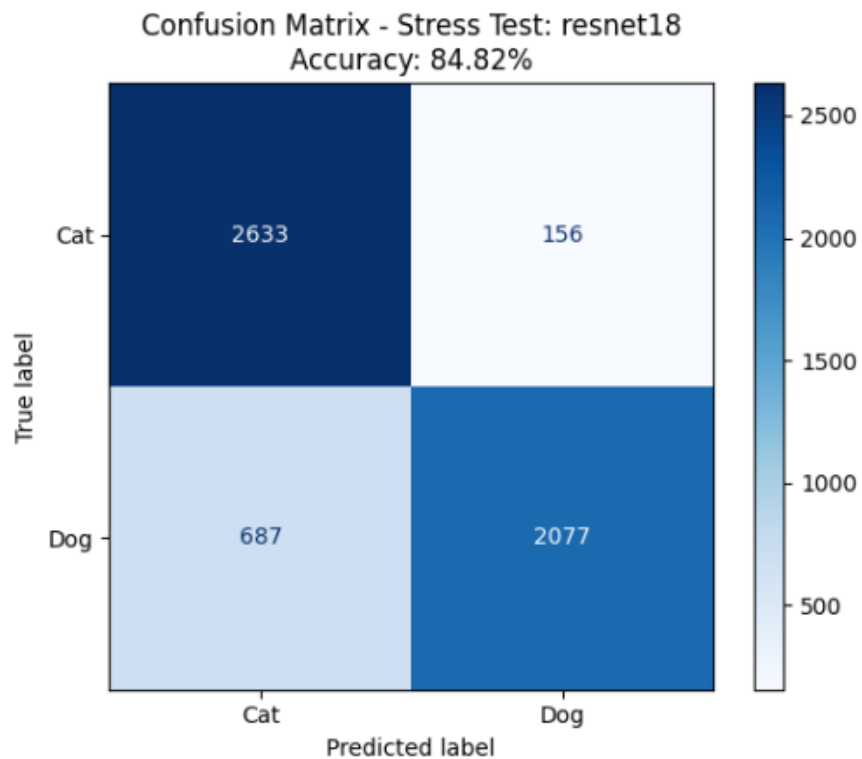


Figure 6: Resnet

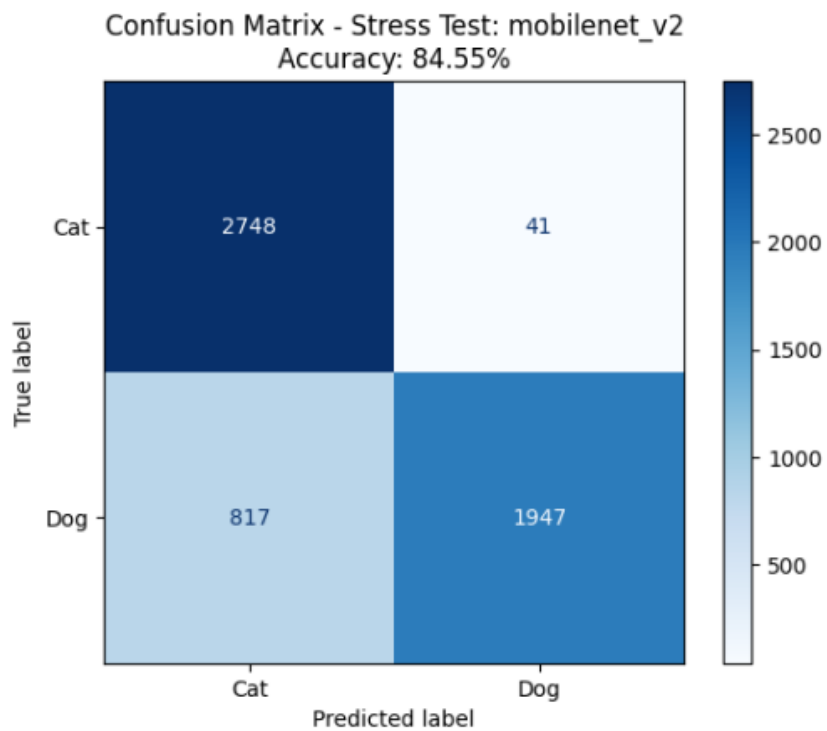


Figure 7: Mobilenet

Entrambe le reti hanno un'accuratezza elevata anche per un set di immagini non ordinate: questo dimostra ancora una volta la capacità di estrarre le feature dalle immagini e di ritrovarle anche in immagini storte o con meno luminosità.

6 Conclusione

Entrambi le CNN offrono delle prestazioni molto alte con l'inserimento dei pesi dai modelli preaddestrati arrivando addirittura a toccare il 98.3 % con sole 5 epoche e circa 10 minuti di addestramento su una GPU Nvidia T4.

Viceversa non rispondono ugualmente quando togliamo i pesi del modello preaddestrato: ma Resnet18 risponde molto meglio arrivando in sole 5 epoche e con un learning rate di 0.001 a 95% di accuratezza contro il 91% di mobilenet_v2.

Infine in un test con le immagini storte e inscurite l'accuratezza si aggira attorno al 84-85% che è molto buona se consideriamo che il nostro dataset di train possedeva solo immagini dritte.

Per eventuali cambiamenti a livello strutturale aggiuntivi delle mie reti che non ho portato nel mio studio con questi comandi si può eventualmente vedere la struttura della rete (numero di layer, filtri ecc) e volendo agire di conseguenza.

```
mio_modello = get_model("mobilenet_v2", num_classes=2) summary(mio_modello, input_size=(3, 224, 224))
```